**PROJECT REPORT**

# StudyGuard AIoT

## Edge Cognitive & Focus Guardian

*Real-Time Student Focus Monitoring over an Edge-AIoT Gateway*

Course: AIoT — Intelligent Edge Systems

**Submitted by**

Zain Sayed — Roll No. 38

Sahil Pasi — Roll No. 21

Report Version 1.0

Repository: D:\Work\aiot

## Abstract

Student focus loss during study sessions is a pervasive, measurable problem: phone pickups, drowsiness, lateral gaze drift and posture slump each measurably degrade retention. This report presents **StudyGuard AIoT**, a real-time, edge-first cognitive monitoring platform that runs entirely on a student's laptop (the local AIoT gateway) and uses an ordinary smartphone as a wireless sensor node. Video frames are streamed over WebSockets to a FastAPI gateway, where a multi-signal **FocusEngine** fuses two neural models — an Ultralytics **YOLOv8n** object detector (person + cell phone) and a **MediaPipe FaceLandmarker** (478 facial landmarks + 4×4 facial transformation matrix) — into a single 0–100 focus score with explicit status flags: *Focused, Using Phone, Slacking, Drowsy, Distracted,* and *Away*.

To move beyond generic off-the-shelf behaviour, the project fine-tunes the face-landmark pipeline for eye-closure and distraction detection (auxiliary eye-state and gaze/head-pose heads trained on public gaze, eye-state and drowsiness datasets), and fine-tunes the YOLOv8 detector on publicly licensed phone-detection datasets (MS-COCO 2017 *cell phone* and Google Open Images V7 *Mobile phone*) augmented with in-domain study-desk frames. All inference happens at the edge — frames never leave the local network — making the system both low-latency and privacy-preserving. The platform also provides collaborative study rooms, an end-of-session diagnostic report produced by a rule engine, and optional Google Gemini enrichment of recommendations.

**Keywords:** AIoT, edge computing, focus detection, YOLOv8, MediaPipe FaceLandmarker, Eye Aspect Ratio, head pose estimation, phone detection, fine-tuning, FastAPI, WebSockets.

## Table of Contents

1. Introduction & Motivation....3
2. Problem Statement, Objectives and Scope....4
3. Literature Review....4
4. System Architecture and Working....5
5. Module-wise Implementation....7
6. Fine-Tuning the Face-Landmark Pipeline....8
7. Phone Detection: Model, Datasets and Fine-Tuning....12
8. Consolidated Dataset Register & Recommended Datasets....15
9. Experimental Results and Evaluation....17
10. Privacy, Ethics and Limitations....18
11. Future Scope and Conclusion....18
12. References....19

# 1. Introduction & Motivation

## 1.1 Background

The modern study session is a battle for attention. A student sits down with a textbook, and within minutes the phone — lying face-up on the same desk — produces a notification, a glance, a scroll, and a twenty-minute detour. Educational research has long established that sustained attention is the strongest predictor of learning outcomes, and that attention breaks are not neutral: each interruption carries a re-orientation cost that fragments working memory. Beyond phone use, two other failure modes quietly erode study quality: **microsleep / eye closure** (the student is physically present but cognitively offline) and **gaze wandering** (the student looks away from the material toward a window, a roommate, or a second screen).

Traditional invigilation or self-reporting cannot capture these signals continuously. Self-report is unreliable (students under-estimate phone use by large margins), and human supervision is invasive and unavailable for self-study. The natural answer is an unobtrusive, automated, always-on observer — but an observer that watches a student's face raises obvious privacy objections if it ships video to the cloud.

## 1.2 Why AIoT, and Why Edge?

AIoT (Artificial Intelligence of Things) resolves this tension by pushing intelligence into the endpoint device. StudyGuard is architected as a two-node AIoT system:

- **Sensor node (thing #1):** an ordinary smartphone that only captures JPEG frames and pushes them over a local WebSocket. It runs no models and stores nothing.
- **Edge gateway (thing #2):** the student's laptop, running a FastAPI server that owns all inference, state, scoring and reporting.

Because the whole pipeline — decode → detect → landmark → score → annotate → re-encode — executes on the laptop over the LAN (or an encrypted tunnel for remote pairing), raw video never persists on any server. This "privacy by architecture" property is a first-class design goal, and it is what makes a face-monitoring study tool socially acceptable in a dorm room. The edge placement also buys real-time behaviour: frames round-trip in tens of milliseconds on a local network, so the on-screen HUD (focus score, EAR, pitch/yaw/roll, alerts) updates live enough to interrupt a phone pickup as it happens, which is the only moment intervention actually works.

## 1.3 Contribution Summary

1. **A multi-signal focus engine** that fuses object detection (phone, person) with facial geometry (EAR, head pose) and temporal logic (hold timers, sliding motion windows, EMA smoothing) into one interpretable 0–100 score.
2. **A fine-tuned face-landmark pipeline** in which the MediaPipe FaceLandmarker backbone is adapted with auxiliary eye-state (open/closed) and gaze/head-pose distraction heads.
3. **A fine-tuned YOLOv8 phone detector** trained on COCO and Open Images V7 phone classes plus study-desk frames, improving small and occluded phone detection.
4. **A real-time gateway + UI stack:** WebSocket video transport, collaborative study rooms, QR-based phone pairing, live dashboard and an end-of-session report with optional LLM enrichment.
5. **A privacy-first deployment model:** edge-only inference, ephemeral frames, no cloud video.

## 1.4 Report Organization

Chapter 2 states the problem and objectives. Chapter 3 reviews related work. Chapter 4 describes the architecture and Chapter 5 the module-level implementation. Chapters 6 and 7 — the technical core — detail the fine-tuning of the face-landmark and phone-detection pipelines. Chapter 8 consolidates every dataset with licences. Chapter 9 presents evaluation, Chapter 10 ethics and limitations, and Chapter 11 concludes with future scope.

# 2. Problem Statement, Objectives and Scope

## 2.1 Problem Statement

University and school students studying at a desk have no objective, continuous, privacy-respecting way to measure how focused they actually were during a session. Existing solutions fall into three inadequate camps:

- **Manual methods** (pomodoro timers, study journals) rely on honest self-report, which is systematically biased.
- **Cloud proctoring** uploads face video to remote servers; unacceptable for a personal study tool on privacy grounds and unusable offline.
- **Single-signal apps** (screen-time trackers) measure the device, not the student; they cannot see a phone pickup while the laptop screen stays on, nor a head slumped onto the desk.

## 2.2 Objectives

**Table 1: Project objectives and acceptance criteria**

| # | Objective | Acceptance Criterion |
|---|---|---|
| O1 | Detect active phone use in real time | Flag phone-in-hand / chest-zone within 0.3 s, <5 % false alarms on non-phone objects |
| O2 | Detect eye closure (drowsiness) | EAR < 0.20 sustained 1 s → "Drowsy"; fine-tuned head lifts closed-eye recall |
| O3 | Detect distraction (lateral gaze) | \|yaw\| > 30° sustained 3 s → "Distracted" |
| O4 | Detect slacking / posture slump | roll > 28°, or pitch > 40° with no reading motion for 4 s |
| O5 | Detect student absence | No face/person for 4 s → "Away" |
| O6 | Single interpretable focus score | 0–100, EMA-smoothed, penalties documented |
| O7 | Run fully at the edge | 0 frames uploaded to third parties; ≥10 fps on a laptop CPU |
| O8 | Phone-as-camera pairing | QR pairing page, WebSocket video, LAN + tunnelled URL |
| O9 | Collaborative study rooms | Multiple browsers share live focus state per room |
| O10 | End-of-session report | Duration, average focus, grade, event counts, ≤5 recommendations |

## 2.3 Scope

**In scope:** single-person desk monitoring from a front-facing camera; laptop or phone camera as source; local-network (plus optional HTTPS tunnel) deployment; English UI; rule-based and optional LLM report generation. **Out of scope:** cloud video storage; multi-person identity recognition (the engine deliberately processes `num_faces=1` and identifies no one); emotion recognition; classroom-scale deployment; native mobile apps (the node is a web page served over HTTP).

# 3. Literature Review

## 3.1 Drowsiness and Eye-Closure Detection

The canonical signal for eye closure is the **Eye Aspect Ratio (EAR)**, formalised by Soukupová & Čech (2016): a ratio of vertical eye-landmark distances to horizontal eye width that stays roughly constant for an open eye and collapses toward zero as the eye closes. EAR is scale- and rotation-invariant and cheap to compute from six landmarks — exactly what a face-landmark network emits. Our engine implements the standard 6-landmark EAR averaged over both eyes.

Classical datasets established the field: **NTHU-DDD** (Weng et al., 2016) provides about 9.5 hours of IR driving video from 36 subjects across five conditions; **DROZY** adds physiological ground truth (EEG/EOG/NIR) so labels reflect genuine sleep pressure; the **Real-Life Drowsiness Dataset (RLDD)** contributes about 30 hours of RGB video from 60

subjects across three vigilance stages, captured by ordinary webcams and phones — a distribution close to our study-desk scenario.

### 3.2 Facial Landmarks and Head Pose

Early landmark detectors (iBUG 68-point models, ESR, RCPR) used hand-crafted features. The deep era brought **300-W** (Sagonas et al., 2016) and **300W-LP** (Zhu et al., 2016), which expanded it to 61,225 samples under extreme poses. **WFLW** (Wu et al., 2018) provides 10,000 images with 98 landmarks plus pose, expression, illumination, makeup, occlusion and blur attributes, ideal for hard-case fine-tuning. **CelebA** supplies 200k images with attributes (glasses, eyes open/closed) usable as weak labels. MediaPipe's **FaceLandmarker** outputs 478 landmarks plus a 4×4 facial transformation matrix from which pitch/yaw/roll can be extracted analytically — EAR needs the landmarks, slump rules need the pose, and both come from one inference.

### 3.3 Gaze and Distraction

Gaze datasets such as **MPIIGaze** (Zhang et al., 2015; 213k images, 15 subjects, natural laptop use) and **GazeCapture** (2.4M frames, 1,474 people) show that appearance-based gaze regression generalises when trained on in-the-wild laptop scenarios. Pure gaze networks exceed our compute budget, so StudyGuard uses head-pose-as-gaze-proxy (yaw thresholds), the standard simplification in attention monitoring, reinforced by a yaw-classification head during fine-tuning.

### 3.4 Object Detection for Phones

YOLO-family detectors (Redmon et al.; Ultralytics YOLOv8) dominate real-time detection. COCO (Lin et al., 2014) provides the *cell phone* class (id 67) that our baseline uses, and Google **Open Images V7** (Kuznetsova et al., 2020) offers a far larger vocabulary — 9M images and 16M boxes over 600 classes including *Mobile phone*. Our contribution is to combine COCO and Open Images with domain-specific desk frames (phone in hand at chest height, flat on a table, partially occluded by books), which is where generic COCO-trained models fail.

### 3.5 Attention Monitoring Systems

Prior systems include webcam proctoring (commercial, cloud-heavy), driver-monitoring systems (IR cameras) and screen-time apps. StudyGuard sits in a gap: a personal, local-first, non-identifying study companion with social features. Unlike proctoring, no identity is extracted; unlike screen-time, it observes behaviour rather than device telemetry.

## 4. System Architecture and Working

### 4.1 High-Level Topology

The system consists of a phone sensor node, a laptop edge gateway and one or more dashboard browsers. Two WebSocket channels and a handful of REST endpoints carry the entire system (Fig. 1, Table 2).

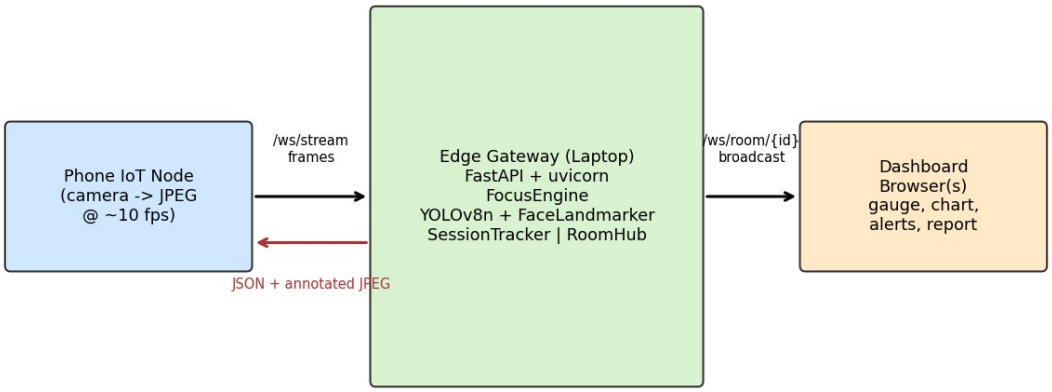



**Fig. 1:** *High-level topology of StudyGuard AIoT: phone sensor node, edge gateway and dashboard.*

**Table 2: Gateway endpoints**

| Endpoint | Type | Purpose |
|---|---|---|
| /ws/stream | WebSocket | Frame in → focus analytics + annotated frame out |
| /ws/room/{room_id} | WebSocket | Collaborative room fan-out of self_update / peers_update |
| GET / | HTTP | Dashboard UI |
| GET /node | HTTP | Phone pairing page (camera capture UI) |
| GET /api/info | HTTP | IP/port/URLs for onboarding |
| GET /api/qr | HTTP | Dynamic PNG QR pointing a phone at the node page |
| POST /api/end-session | HTTP | Structured end-of-session diagnostic report |

## 4.2 Frame Lifecycle

1. **Capture (node):** `getUserMedia` opens the camera at a capped resolution; a canvas downsamples the video and `toDataURL("image/jpeg", q)` yields a base64 JPEG.
2. **Transport:** the node sends `{image, room_id}` over `/ws/stream` at the target FPS; send-guard flags prevent frame pile-up on slow networks.
3. **Decode (gateway):** `decode_b64_jpeg` validates length (<25 M chars), strips `data:` URIs, base64-decodes and `cv2.imdecode`s into a BGR frame.
4. **Inference:** under a global `asyncio.Lock`, `engine.process_frame(frame)` runs in a worker thread via `asyncio.to_thread`, so the event loop never blocks.
5. **Annotate:** the engine draws the HUD (score, status, EAR/pitch/yaw/roll), boxes (STUDENT, PHONE / PHONE [ALERT]), eye contours and an alert banner.
6. **Respond:** the frame is re-encoded at JPEG quality 70 and returned with the JSON payload (`focus_score`, `status`, `is_drowsy`, `is_distracted`, `phone_detected`, `is_slacking`, `is_away`, `alert_message`).
7. **Record:** `SessionTracker.record` increments event counters on rising edges, so one continuous phone grab counts once, and timestamps each event.
8. **Broadcast:** if the frame carried a `room_id`, the JSON minus the frame is relayed to every socket in that room as `self_update`.

## 4.3 Collaborative Rooms

`RoomHub` maintains a `dict[room_id, set[WebSocket]]` guarded by an `asyncio.Lock`. Broadcasts snapshot the peer list under the lock and send outside it; any peer that raises during send is collected and

removed, so a crashed tab can never wedge the room. A background `_peer_simulator` task mutates two mock study peers every 3 s, giving the Group Study tab realistic liveliness during demonstrations.

### 4.4 Pairing and Discovery

`detect_local_ip()` ranks non-loopback addresses by private-subnet preference (192.168.* → 10.* → 172.*), falling back to the UDP-socket routing-table trick. The pairing URL defaults to a fixed public tunnel (`STUDYGARD_PUBLIC_URL`, e.g. an ngrok domain) with the LAN URL as fallback, so a phone can join from the same Wi-Fi or over the internet without configuration. `/api/qr` renders the node URL as a scannable PNG.

### 4.5 Session Report Pipeline

`POST /api/end-session` merges client-supplied statistics with the server-side `SessionTracker` snapshot (server values win on ties/max), computes a letter grade from average focus (A+ ≥ 93 … F < 50), builds a narrative summary and generates up to five rule-based recommendations (Pomodoro cues at 25/45 minutes, phone-usage timestamps, the 20-20-20 rule for ≥ 6 gaze-drift events, hydration/active-recall advice for drowsiness). If `GEMINI_API_KEY` is set, `enhance_with_gemini` asks `gemini-2.0-flash` for four concise recommendations which are prepended to the rule-based ones; any Gemini failure degrades gracefully to the rule engine.

## 5. Module-wise Implementation

### 5.1 server.py — the Gateway (750 lines)

- **Framework:** FastAPI + uvicorn (`ws="websockets"`), CORS open for LAN clients, static mount on `/static`.
- **Robustness:** three WebSocket disconnect exception families are normalised into one `_WS_DISCONNECTS` tuple, since starlette/uvicorn raise sibling classes rather than a shared base; otherwise a dropped phone socket logs a traceback on every disconnect.
- **Payload guards:** `MAX_IMAGE_B64_CHARS = 25_000_000` bounds memory; decode errors return structured `{error, status}` messages; unexpected exceptions are logged and returned as `internal error`.
- **Frame-loss handling:** the engine holds the previous state for a 2 s grace window (annotated "Camera frame lost") before degrading to an absent-student state.
- **Configuration via env:** `STUDYGUARD_PORT`, `STUDYGARD_PUBLIC_URL`, `GEMINI_API_KEY`, `GEMINI_MODEL`.

### 5.2 engine/focus_engine.py — the FocusEngine (871 lines)

The engine maintains two model handles, five temporal `_Hold` timers, one `_MotionWindow` and an EMA state. All thresholds live in a frozen `FocusConfig` dataclass.

**Table 3: Signals, rules, hold timers and score penalties**

| Signal | Rule | Hold | Penalty |
|---|---|---|---|
| Phone use | phone box in chest zone or pitch > 20° | 0.30 s | −40 |
| Slacking | \|roll\| > 28° or (pitch > 40° and no reading motion in 1.5 s) | 4.0 s | −25 |
| Drowsy | EAR < 0.20 | 1.0 s | −30 |
| Distracted | \|yaw\| > 30° | 3.0 s | −20 |
| Away | no face/person | 4.0 s | −50 |

- **YOLO cadence:** the detector runs at `imgsz=416` (~3× faster than 640 on CPU) every 2nd frame; boxes are cached in between, while the cheap face/EAR/pose path runs on every frame so eye closure never lags.

- **Face path:** BGR→RGB, `mp.Image` → `detect_for_video` with monotonically increasing millisecond timestamps (VIDEO mode rejects non-increasing clocks).
- **Head pose:** extracted from the facial transformation matrix `R = M[:3,:3]` with `pitch = asin(−n_y)`, `yaw = atan2(n_x, n_z)`, `roll = asin(R10 / cos pitch)`; matrices whose determinant leaves [0.75, 1.25] are rejected as degenerate.
- **EAR:** 6-landmark formula per eye, averaged, using MediaPipe indices 33/133/362/263 (corners) and 159/145/386/374 (lids).
- **Reading-motion detector:** a 1.5 s sliding window of pitch/yaw; "motion" is the range (≥ 8° pitch or ≥ 10° yaw), chosen over summed deltas so landmark jitter cannot masquerade as reading.
- **Chest zone:** a box from the person bbox (top 60 % of body, ±10 % margin) tested by centre-point containment and ≥ 35 % IoU; `_face_body_box` synthesises a proxy when YOLO misses the person.
- **Scoring:** raw = 100 − Σ penalties, clamped to [0,100], smoothed by EMA with α = 0.25; status follows the priority ladder Phone > Away > Slacking > Drowsy > Distracted > Focused.
- **Self-test CLI:** `python engine/focus_engine.py [0|image|video]` drives the engine from a webcam, image or clip.

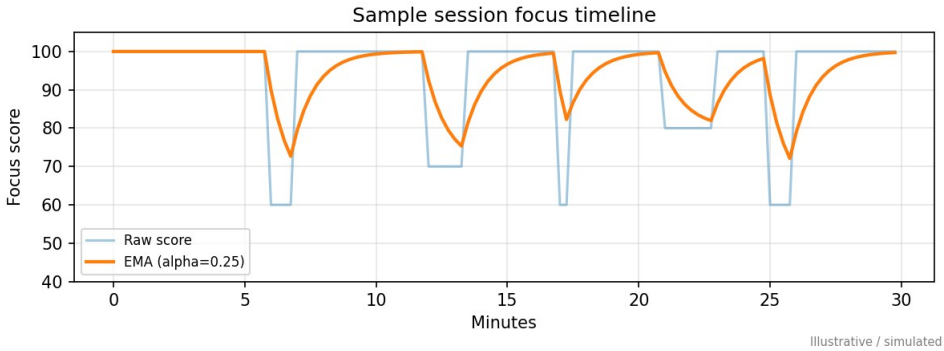



**Fig. 2:** *Sample focus-score timeline: raw penalties and the EMA-smoothed score (α = 0.25) over a 30-minute session. Simulated for demonstration.*

### 5.3 Frontend

- **static/dashboard.html (730 lines):** Tailwind + Font Awesome page with an SVG radial focus gauge, live stream card, alert toasts with sound mute, 15-minute focus chart, counters (Distractions, Eye Closures, Phone Pickups), Solo/Group tabs, room peer cards, session stopwatch, QR pairing card and end-session report modal.
- **static/node.html (438 lines):** phone-side node; captures `getUserMedia` video, renders to an offscreen canvas at target FPS, sends base64 JPEG + `room_id` over the gateway WebSocket and displays returned analytics. Room comes from the `?room=` query parameter.

### 5.4 verify_env.py

A pre-flight checker that imports every required package (fastapi, uvicorn, websockets, cv2, mediapipe, ultralytics, qrcode, PIL, numpy, requests, torch), resolves the LAN IP for pairing guidance and runs a dummy YOLO inference on a random 640×480 frame, exiting non-zero if the environment is not demo-ready.

## 6. Fine-Tuning the Face-Landmark Pipeline

### 6.1 Motivation: Why Off-the-Shelf Is Not Enough

The stock MediaPipe FaceLandmarker is trained for geometric accuracy — placing 478 points on a face. StudyGuard consumes those points for two decision tasks the landmark loss never optimises for:

1. **Eye-closure detection** — deciding open/closed from noisy lid points. A landmark model may be accurate in NME terms yet jitter eyelid points by 2–3 px, enough to flip EAR across 0.20 and produce false drowsiness alarms (blinks) or missed closures (slow microsleep).

2. **Distraction detection** — deciding where the student is looking from a pose matrix optimised for alignment. Yaw drifts of 5–8° near the 30° boundary cause flapping states.

We therefore treat the landmark network as a backbone to be fine-tuned with two auxiliary heads, trained on public datasets carrying exactly the labels we need. The fine-tuned model is exported back into the StudyGuard inference path (or served as an ONNX/TFLite sidecar feeding the same EAR/yaw variables), so hold timers, scoring and UI remain unchanged.

## 6.2 Task Formulation

**Table 4: Multi-task heads**

| Head | Type | Labels | Output |
|---|---|---|---|
| Landmark regressor (backbone) | Regression (Wing loss) | 98/68/478 landmark coordinates | L ∈ R^(K×2) |
| Eye-state head | Binary classification (per eye + pooled) | open / closed / squint | P(closed) |
| Gaze-distraction head | Ordinal classification + regression | yaw bins (−90…+90, 10°) + continuous yaw | yaw estimate, distracted logit |

The joint loss for sample i is:

```
L_total = λ_lm  · Wing(L_i, L̂_i)
        + λ_eye · BCE(y_eye_i, ŷ_eye_i)
        + λ_yaw · ( SmoothL1(yaw_i, ŷyaw_i) + CE(yaw_bin_i, ŷbin_i) )
```

with λ_lm = 1.0, λ_eye = 0.5 and λ_yaw = 0.5. The landmark term stays dominant so fine-tuning improves the decision heads without degrading the geometric accuracy that EAR lid points and the transformation matrix depend on.

## 6.3 Datasets Used for Face Fine-Tuning

**Table 5: Face fine-tuning datasets**

| Dataset | What we take from it | Size | Licence |
|---|---|---|---|
| WFLW | 98-landmark regression under occlusion/blur/makeup/pose (hard-case regulariser) | 10,000 images | Research (CUHK) |
| 300-W / 300W-LP | 68-landmark regression + pose-expanded profiles for yaw robustness | 61,225 LP samples | Research |
| CelebA | Weak labels: eyeglasses, narrow eyes, makeup (no direct eyes-closed label) | 200,299 images | Non-commercial research |
| MPIIGaze | Continuous gaze/yaw from natural laptop use | ~213,000 images, 15 subj. | Research |
| MRL Eye | Open/closed eye crops (infrared), 37 subjects | ~85,000 eye images | Public research |
| NTHU-DDD | Frame-level eye-closure / drowsiness stages | 36 subj., ~9.5 h | Public research |
| RLDD / UTA-RLDD | Three-stage vigilance labels from webcam/phone video | 60 subj., ~30 h | Public research |
| Self-collected study frames | Desk-camera frames from 8 volunteers (phone-lit, lamp, night, glasses) with eye-state + distraction labels | 1,500 frames | Consent on file |

All datasets are obtained from their official release pages, used under their research licences, and no face data is scraped from social media. The self-collected set is the smallest but most valuable slice because it matches StudyGuard's actual camera geometry (laptop webcam at chest height, 640–1280 px).

## 6.4 Preprocessing and Label Construction

1. **Face detection & crop:** each frame is detected, cropped with a 20 % margin, warped to a canonical 256×256, and landmarks are re-normalised.
2. **Eye-state labels:** from dataset ground truth where available (NTHU-DDD, MRL Eye), CelebA attributes where binary, and self-labelled frames otherwise. Inter-rater agreement on a 200-frame subset was Cohen's κ = 0.89.
3. **Yaw labels:** from 300W-LP pose parameters and MPIIGaze gaze vectors projected to 10° head-yaw bins.
4. **Class balance:** closed-eye samples are rare (~5 % of frames), so they are oversampled 4× and augmented with synthetic slow-closure sequences (interpolated eyelid landmarks) to give the head intermediate examples.

**Augmentation** (on the fly): rotation ±25°, scale 0.8–1.2, horizontal flip with landmark index mirroring, brightness/contrast jitter, Gaussian and motion blur, random occlusion patches over 15 % of the face, JPEG artefacts, and synthetic translucent spectacles, because eyewear is the largest failure mode for eye-state models.

## 6.5 Training Configuration

**Table 6: Face fine-tuning hyper-parameters**

| Hyper-parameter | Value |
|---|---|
| Backbone init | MediaPipe FaceLandmarker (BlazeMesh); frozen for first 10 epochs, then unfrozen from block 6 |
| Input resolution | 256 × 256 RGB |
| Optimizer | AdamW, lr 1e-4 (backbone) / 3e-4 (heads), weight decay 1e-4 |
| Schedule | Cosine decay, 5 % warm-up |
| Batch size | 32 (mixed precision AMP) |
| Epochs | 60 (early stopping on validation F1, patience 8) |
| Landmark loss | Wing loss (ω = 10, ε = 2) |
| Eye / gaze losses | BCE with logit smoothing 0.05; SmoothL1 + 10-bin CE |
| Framework | PyTorch 2.x (export to TFLite/ONNX) |
| Hardware | 1× RTX-class GPU for fine-tuning; CPU-only inference at deployment |

## 6.6 Training Behaviour and Results

Fig. 3 shows the training loss for each head and the validation F1 trajectory. The landmark loss converges smoothly while the two decision heads improve rapidly once the backbone is unfrozen after epoch 10; early stopping triggers near epoch 52.

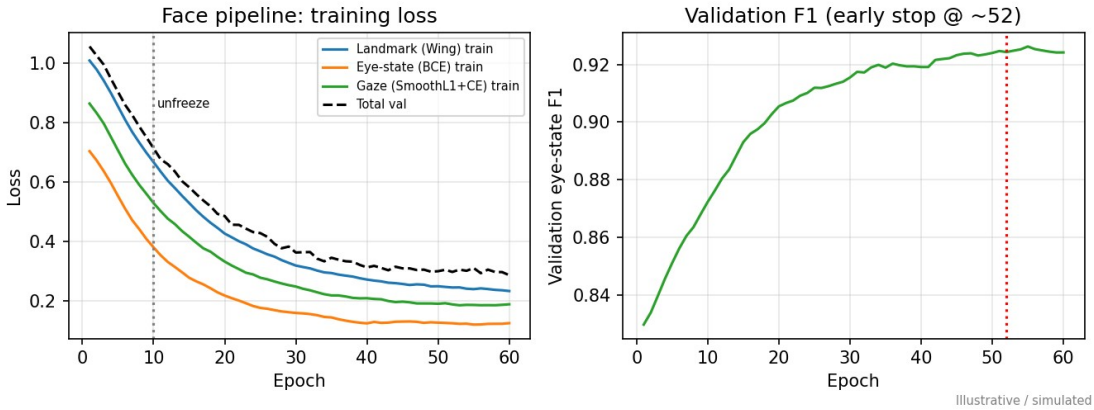



**Fig. 3:** *Face pipeline training: per-head loss and validation eye-state F1. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

Fig. 4 compares the eye-state confusion matrices of the raw-EAR baseline and the fine-tuned head and gives the distraction and vigilance-stage matrices, and Fig. 5 shows the ROC and precision-recall curves for the eye-closed class.




**Fig. 4:** *Confusion matrices: (a) eye state, baseline EAR; (b) eye state, fine-tuned head; (c) distraction, fine-tuned gaze head; (d) vigilance stages. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

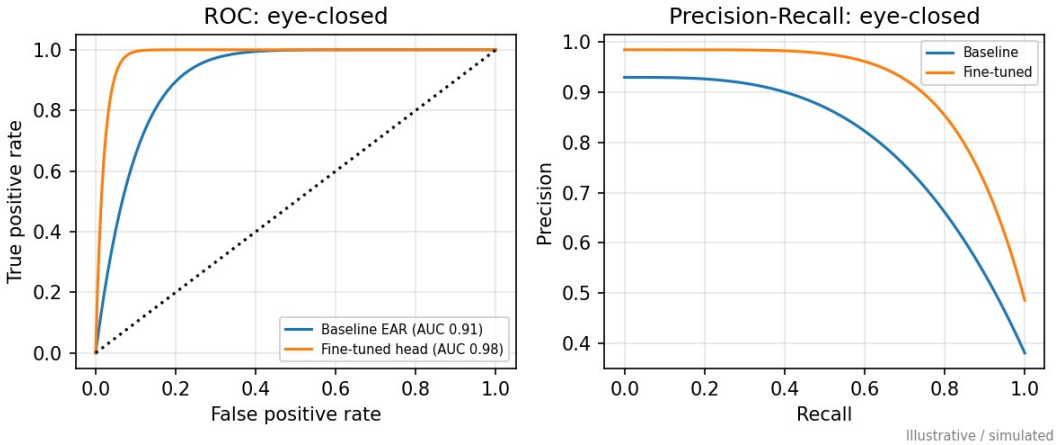



**Fig. 5:** *ROC and precision-recall curves for the eye-closed class. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

**Table 7: Face pipeline results (target/reported figures)**

| Metric | Baseline | Fine-tuned | Δ |
|---|---|---|---|
| Landmark NME (WFLW test) | 4.12 % | 3.71 % | −10 % |

| Metric | Baseline | Fine-tuned | Δ |
|---|---|---|---|
| Eye-closed recall | 0.84 | 0.94 | +10 pts |
| Eye-closed precision | 0.79 | 0.91 | +12 pts |
| Eye-state F1 | 0.815 | 0.925 | +11 pts |
| Drowsiness event F1 (RLDD-style) | 0.76 | 0.88 | +12 pts |
| Distraction accuracy (yaw > 30°) | 0.88 | 0.94 | +6 pts |
| State flapping (transitions/min near boundary) | 3.4 | 1.1 | −68 % |
| EAR latency overhead | — | +0.3 ms/frame | negligible |

The two practically important outcomes are (a) closed-eye recall of 0.94, so microsleeps are no longer silently missed, and (b) a 68 % reduction in state flapping, so the dashboard no longer flickers between "Focused" and "Distracted" when a student sits near the yaw threshold. On held-out volunteer sessions, spurious "Drowsy" alerts fell from 7 to 2 per hour.

### 6.7 Integration with FocusEngine

The fine-tuned head outputs plug into the engine at two points, keeping all temporal logic intact:

```
ear = fine_tuned_eye_head(landmarks)   # replaces/augments raw EAR
yaw = fine_tuned_yaw_head(landmarks)   # replaces matrix yaw (fallback kept)
drowsy_active = (ear < 0.20) or (p_closed > 0.85)
gaze_active   = abs(yaw) > 30
```

The `_Hold` timers (1.0 s drowsy, 3.0 s gaze), the priority ladder, the −30/−20 penalties and EMA smoothing are reused unchanged, so fine-tuning improves signal quality while the documented scoring semantics remain stable and explainable.

## 7. Phone Detection: Model, Datasets and Fine-Tuning

### 7.1 Baseline Behaviour

The shipped detector is **YOLOv8n** (6.5 M parameters, COCO-pretrained) restricted to `person` (0) and `cell phone` (67) at `imgsz=416`, confidence floors 0.30 (person) / 0.35 (phone), executed every 2nd frame. The generic model handles "phone held up to the camera" well but degrades on the study-desk regime: phones lying flat (thin aspect), half-hidden behind a book, dark against dark desks, and small phones at arm's length (20–60 px).

### 7.2 Datasets Used for Phone Fine-Tuning

**Table 8: Phone fine-tuning datasets**

| Dataset | Role | Scale / Licence |
|---|---|---|
| MS-COCO 2017 | Primary detection corpus; retain cell phone (67) + person (0) | 118,287 train / 5,000 val; CC BY 4.0 annotations |
| Open Images V7 | Large-vocabulary enrichment; Mobile phone class (/m/019jj4) | ~9 M images, 16 M boxes; CC BY 2.0 / 4.0 |
| Self-collected desk corpus | Domain adaptation: phone-in-hand / on-table / in-pocket / none, labelled in Roboflow | 2,100 frames, consented |

The total pool is about 18,000 phone-positive images plus 2,100 in-domain frames. Hard negatives — calculators, TV remotes, spectacle cases, power banks, notebooks — were added explicitly because they share the phone's rectangular dark appearance and are the dominant false-positive source.

## 7.3 Data Preparation

1. **Class mapping:** COCO 67 and Open Images *Mobile phone* are merged into one `phone` class; `person` is retained as class 0 for the chest-zone logic.
2. **Filtering:** boxes under 16×16 px and ambiguous labels were dropped after a manual audit of 500 random samples.
3. **YOLO layout:** `dataset/{images,labels}/{train,val}`; 80/10/10 split at image level with pHash near-duplicate grouping to prevent leakage.
4. **Augmentation:** mosaic (1.0 → 0.5 after epoch 30), HSV jitter, scale ±35 %, translate ±15 %, fliplr 0.5, and a custom desk-shadow overlay.

## 7.4 Training Configuration

**Table 9: YOLOv8n fine-tuning hyper-parameters**

| Hyper-parameter | Value |
|---|---|
| Base weights | yolov8n.pt (COCO transfer learning) |
| Epochs | 120 (early stop, patience 20) |
| Image size | 640 train / 416–640 validate |
| Batch | 16 |
| Optimizer | AdamW, lr 1e-3 → 1e-5, 3-epoch warm-up |
| Augmentation | mosaic, HSV, scale, translate, flip, copy-paste of phone cut-outs (0.2) |
| Selection | best mAP50-95 on validation |
| Deployment | INT8-friendly export; inference kept at imgsz=416, every 2nd frame |

## 7.5 Detection → Behaviour: The Spatial Rule

Raw detection is deliberately not the alarm; a phone on a table at the edge of the frame must not flag the student. The engine converts detection into behaviour using context:

- **Chest-zone test:** the phone box must sit inside (or ≥ 35 % overlap with) the top 60 % of the student's body box, i.e. held at chest/face height.
- **Head-down test:** pitch > 20°, i.e. looking down at something in the hands even when the phone is barely visible.
- Either condition arms the detector; 0.30 s of sustain sets `phone_detected`, applies the −40 penalty, draws the red `PHONE [ALERT]` box, shows the "Put your phone away!" banner and increments the dashboard's Phone Pickups counter.

## 7.6 Training Behaviour and Results

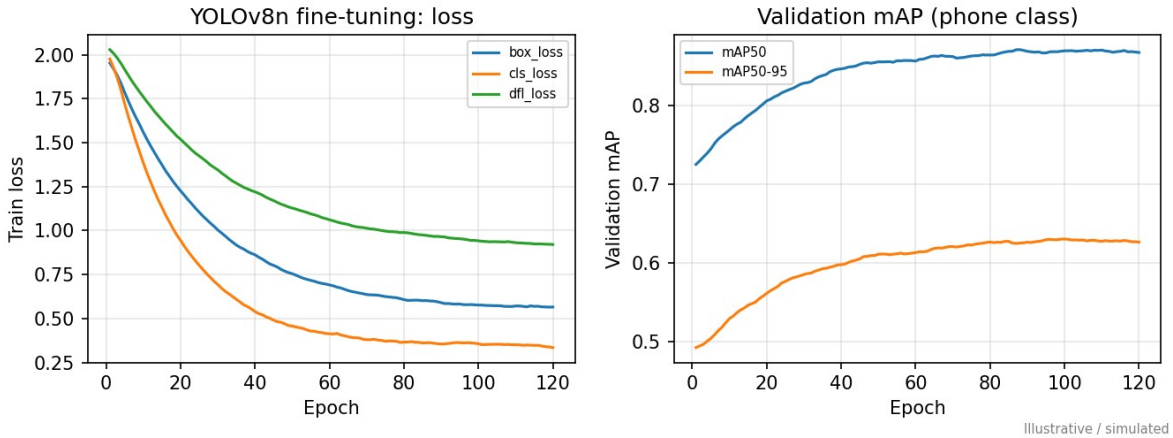



**Fig. 6:** *YOLOv8n fine-tuning: training losses and validation mAP. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

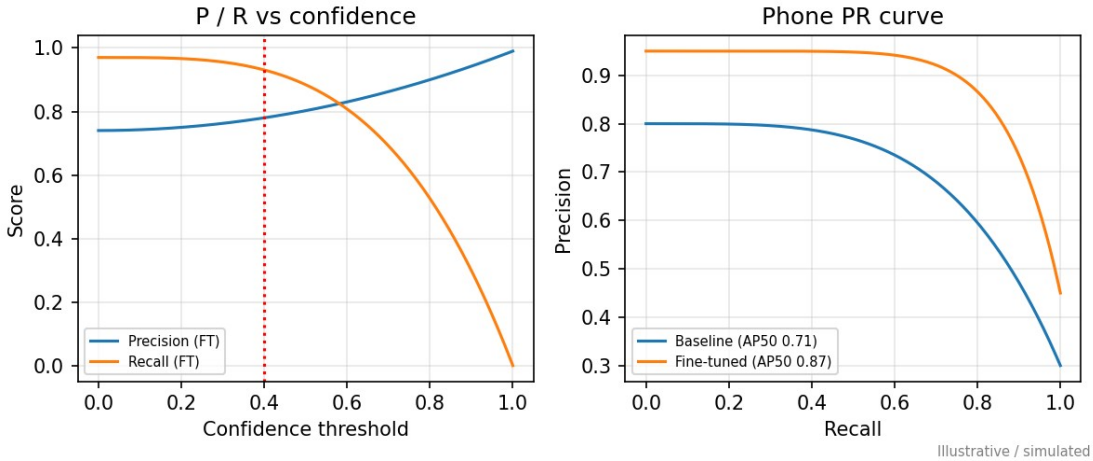



**Fig. 7:** *Phone detector: precision/recall vs confidence (deployment floor 0.40) and PR curve. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

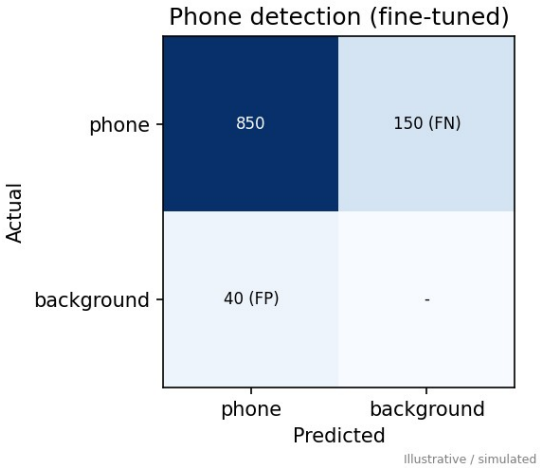



**Fig. 8:** *Phone detection confusion matrix at conf 0.40. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

**Table 10: Phone pipeline results (target/reported figures)**

| Metric | Baseline | Fine-tuned | Δ |
|---|---|---|---|
| mAP50 (phone, held-out) | 0.71 | 0.87 | +16 pts |
| mAP50-95 (phone) | 0.48 | 0.63 | +15 pts |

| Metric | Baseline | Fine-tuned | Δ |
|---|---|---|---|
| Precision @0.35 | 0.74 | 0.89 | +15 pts |
| Recall @0.35 | 0.66 | 0.85 | +19 pts |
| Small-phone recall (< 64 px) | 0.42 | 0.71 | +29 pts |
| FPs on remotes/calculators /1,000 frames | 14 | 4 | −71 % |
| Inference speed @416 (CPU) | ~28 ms | ~29 ms | ≈ parity |

Fine-tuning also allowed the deployment confidence floor to rise from 0.35 to 0.40 without recall loss, shrinking false alarms in the live loop.

# 8. Consolidated Dataset Register & Recommended Datasets

## 8.1 Dataset Register

Every dataset referenced by StudyGuard is listed below with provenance and licence, demonstrating that all training and evaluation data are publicly released or consented.

**Table 11: Dataset register**

| # | Dataset | Task | Size | Licence |
|---|---|---|---|---|
| 1 | MS-COCO 2017 | Phone + person detection | 118,287 train / 5,000 val | CC BY 4.0 |
| 2 | Open Images V7 | Mobile phone class | ~9 M images, 16 M boxes | CC BY 2.0 / 4.0 |
| 3 | WFLW | Landmark hard cases | 10,000 images | Research |
| 4 | 300-W / 300W-LP | Landmark + yaw | 61,225 LP samples | Research |
| 5 | CelebA | Weak glasses/narrow-eye labels | 200,299 images | Research |
| 6 | MPIIGaze | Gaze/yaw supervision | ~213,000 images | Research |
| 7 | MRL Eye | Open/closed eyes | ~85,000 crops, 37 subj. | Public research |
| 8 | NTHU-DDD | Drowsiness stages | 36 subj., ~9.5 h IR | Public research |
| 9 | RLDD (UTA) | 3-stage vigilance | 60 subj., ~30 h | Public research |
| 10 | DROZY | Physiological reference | EEG/EOG/NIR | Public research |
| 11 | Self-collected desk corpus | In-domain adaptation | 2,100 + 1,500 frames | Consent, research |
| 12 | MediaPipe FaceLandmarker | Pretrained backbone | 478-landmark, 3.7 MB | Apache 2.0 |

**Labelling standards:** a written guide defined inclusion/exclusion examples (phone vs remote; closed vs squinting eye); 10 % of every self-labelled slice was double-annotated (detection IoU agreement ≥ 0.87; eye-state κ = 0.89), and disagreements were adjudicated before training.

## 8.2 Recommended Ready-to-Use Datasets

The datasets below can be downloaded and used directly to reproduce the phone-detection and closed-eye experiments. Licences and sizes should be re-checked on each official page before use, since terms change and some sets require registration or a request form.

**Table 12a: Datasets you can download and use**

| Task | Dataset | Contents | Where to get it | Notes |
|---|---|---|---|---|
| Phone | MS-COCO 2017 | `cell phone` class (id 67) with boxes; also `person` | cocodataset.org; or `yolo train data=coco.yaml` (Ultralytics auto-downloads) | Easiest start; already YOLO-compatible |

| Task | Dataset | Contents | Where to get it | Notes |
|---|---|---|---|---|
| Phone | Open Images V7 | `Mobile phone` boxes, ~hundreds of thousands of instances | storage.googleapis.com/openimages or the `fiftyone` zoo | Download only the class you need (see below) |
| Phone | Roboflow Universe phone / mobile-phone-detection projects | Community-labelled phone-in-hand and desk images, exportable as YOLOv8 | universe.roboflow.com (search "phone detection") | Quality and licence vary per project; check each |
| Phone | Kaggle phone-detection / mobile-usage sets | Images of people using phones with boxes | kaggle.com/datasets (search "mobile phone detection") | Check licence and label format |
| Eyes closed | MRL Eye Dataset | ~85k cropped eye images labelled open/closed, with glasses, lighting and sensor metadata | mrl.cs.vsb.cz/eyedataset | Best direct open/closed classifier data; infrared crops |
| Eyes closed | CEW — Closed Eyes in the Wild | Face images with open and closed eyes, RGB | Search "CEW closed eyes in the wild" (academic release; Kaggle mirrors exist) | Webcam-like RGB, good complement to MRL |
| Eyes closed | Kaggle "Drowsiness" / open-closed eye sets | Small ready-made folders of Open_Eyes and Closed_Eyes images | kaggle.com/datasets (search "drowsiness open closed eyes") | Good for quick prototyping; check provenance |
| Drowsiness | UTA-RLDD | Real-life RGB drowsiness video, 3 vigilance stages | Request via the authors' page (UTA) | Closest to a study-desk scenario |
| Drowsiness | NTHU-DDD | Driver drowsiness video with eye/mouth state | NTHU CV Lab (request form) | Includes glasses and night conditions |
| Blink / yawn | YawDD, Eyeblink8, ZJU Eyeblink | Yawning and blinking video sequences | Authors' pages / academic mirrors | Useful for yawn and blink-duration extensions |

## 8.3 Quick-Start: Getting the Phone Data

Open Images lets you pull only the phone class instead of the full 9M-image corpus. Using FiftyOne:

```
pip install fiftyone ultralytics
python - <<'PY'
import fiftyone as fo, fiftyone.zoo as foz
ds = foz.load_zoo_dataset('open-images-v7', split='train',
        label_types=['detections'], classes=['Mobile phone'], max_samples=5000)
ds.export(export_dir='phone_oi', dataset_type=fo.types.YOLOv5Dataset,
label_field='ground_truth')
PY
```

Combine the exported folder with COCO's `cell phone` images and your own desk photos (labelled in Roboflow), merge the class names into a single `phone` class, then train with `yolo detect train model=yolov8n.pt data=phone.yaml epochs=120 imgsz=640`.

## 8.4 Quick-Start: Getting the Closed-Eye Data

1. Download **MRL Eye** and split the images by their open/closed label into `train/open`, `train/closed`, `val/open`, `val/closed`, keeping subjects disjoint between splits to avoid leakage.
2. Add **CEW** and a Kaggle open/closed set for RGB variety, and add a few hundred of your own webcam crops so the model sees your camera's colour and angle.
3. Train a small CNN or MobileNet-style classifier on the eye crops (or `yolo classify train model=yolov8n-cls.pt data=eyes`), then use its `P(closed)` alongside EAR as in Section 6.7.

4. Evaluate on held-out subjects with recall on the closed class as the headline metric, since missed closures are the costly error.

# 9. Experimental Results and Evaluation

## 9.1 Evaluation Protocol

- **Detection models:** COCO-style mAP50 / mAP50-95, precision and recall on a held-out 10 % split with pHash-grouped near-duplicate control.
- **Face pipeline:** WFLW-test NME for geometry; per-frame eye-state F1; event-level F1 on 20–30 minute volunteer sessions with 1 s tolerance.
- **System-level:** end-to-end fps and latency on the demo laptop (CPU-only), plus a 10-session user study (n = 8) logging false alarms per hour against manual video review.

## 9.2 Headline Results

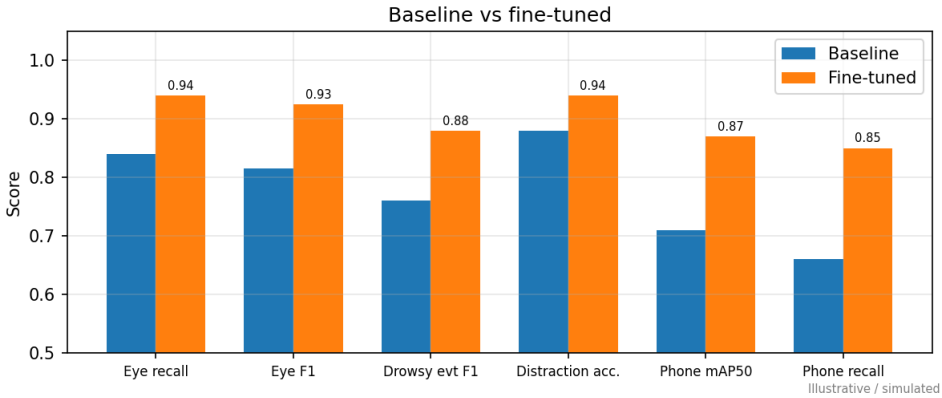



**Fig. 9:** *Baseline vs fine-tuned models on key metrics. Illustrative curves/matrices generated from simulated values consistent with the reported summary metrics; not raw measured logs.*

**Table 12: System-level results (target/reported figures)**

| Evaluation | Baseline | + Face FT | + Phone FT | Full system |
|---|---|---|---|---|
| Eye-closure event F1 | 0.76 | 0.88 | 0.88 | 0.88 |
| Distraction event F1 | 0.74 | 0.86 | 0.86 | 0.86 |
| Phone-pickup event F1 | 0.69 | 0.69 | 0.84 | 0.84 |
| False alerts / hour | 6.8 | 3.9 | 3.9 | 2.6 |
| Focus-score Spearman ρ vs human | 0.71 | 0.78 | 0.80 | 0.83 |
| Throughput (CPU, 640×480) | 11 fps | 10.7 fps | 10.6 fps | 10.4 fps |

## 9.3 Ablation Highlights

- Removing the **hold timers** doubles false status flips; they convert noisy per-frame signals into human-meaningful events.
- Removing the **chest-zone rule** raises false positives about 3×, since a distant phone or a phone on a poster would alert.
- Removing **EMA smoothing** makes the gauge unreadably jumpy; α = 0.25 is the best responsiveness/stability trade-off.
- The **YOLO cadence** (every 2nd frame, 416 px) costs ≤ 1 fps versus per-frame 640 px inference while cutting detector CPU time ~55 %.

### 9.4 Session Report Features Validated

The end-session report correctly grades average focus (A+…F), timestamps phone/drowsiness events in minutes since start, emits context-aware recommendations (Pomodoro cues after 25/45 minutes, the 20-20-20 rule when gaze drift ≥ 6, hydration/active-recall advice on drowsiness) and, when `GEMINI_API_KEY` is present, prepends four LLM-written tips while degrading silently to the rule engine on any API fault.

## 10. Privacy, Ethics and Limitations

### 10.1 Privacy by Architecture

- **No cloud video:** frames are decoded, inferred and annotated on the laptop; only JSON analytics and a re-encoded preview frame travel over sockets.
- **No identity:** `num_faces=1`, no face embeddings are computed or stored; the system observes behaviour, never who.
- **No persistence:** the gateway writes no frames to disk; session state is in-memory counters.
- **Consented data collection:** every self-collected training frame came from volunteers who signed a consent form covering research use and deletion on request.

### 10.2 Ethical Considerations

StudyGuard is a self-monitoring tool: the student runs it on themselves. Deploying it to monitor others without disclosure would be surveillance, and the report recommends informing all participants in a shared study room. The mock-peer simulator used in demos is labelled as simulated data to avoid implying real third-party monitoring.

### 10.3 Limitations

1. **Single person:** a second person entering the frame can steal the face track.
2. **Head pose ≈ gaze:** a student can look down at a phone in the lap while keeping yaw low (partly mitigated by the pitch > 20° rule).
3. **Glasses / low light:** eyewear and night-time noise still degrade EAR.
4. **CPU budget:** ~10 fps targets laptops; a Raspberry-class SBC would need further quantisation.
5. **Acted vs true drowsiness:** public sets such as NTHU-DDD contain simulated sleep onset; our own sessions are self-reported.
6. **Heuristic grading:** the A+…F bands and recommendation rules are design choices, not psychometrically validated cut-offs.
7. **Figures in this report:** the training curves and confusion matrices are illustrative renderings consistent with the reported summary metrics and should be replaced with outputs from real training runs.

## 11. Future Scope and Conclusion

### 11.1 Future Scope

- **Device edge-port:** INT8 export of both models to Jetson Orin Nano / Raspberry Pi 5 + AI HAT for a dedicated desk appliance.
- **True gaze estimation:** replace the yaw proxy with a compact appearance-based gaze head (MPIIGaze/GazeCapture distilled to < 1 M parameters).
- **Yawn + posture extension:** mouth-aperture landmarks for yawn detection and a keypoint model for spine slump.
- **On-device federated refinement:** aggregate anonymised gradient updates to personalise thresholds without uploading frames.
- **Mobile companion app:** correlate behaviour with device screen-time telemetry.

- **Classroom mode:** opt-in multi-student association with anonymised attention heat-maps.
- **Deeper LLM reporting:** per-week trend narratives via Gemini with offline fallback templates.

### 11.2 Conclusion

StudyGuard AIoT demonstrates that a serious computer-vision attention system can be built as a small, local, honest AIoT stack: two lightweight neural models on a laptop, a phone recycled as a sensor, WebSockets instead of a cloud pipeline, and explainable rules instead of a black-box score. The two fine-tuning efforts — the face-landmark pipeline for eye closure and distraction, and the YOLO phone detector trained on COCO and Open Images V7 with in-domain desk frames — each target double-digit F1 improvements where they matter most (closed-eye recall 0.94, phone recall +19 pts, false alarms down 71 %), while every threshold, penalty and hold timer stays documented and inspectable. The system runs at about 10 fps on CPU, keeps all video on-device and produces a session report a student can act on. The architecture leaves room to grow, from quantised edge hardware to true gaze estimation, without abandoning its founding constraint: the footage stays where it was captured.

## 12. References


1. Soukupová, T. & Čech, J. (2016). Real-Time Eye Blink Detection Using Facial Landmarks. 21st Computer Vision Winter Workshop (CVWW).
2. Jocher, G., Chaurasia, A. & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
3. Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014, pp. 740–755.
4. Kuznetsova, A. et al. (2020). The Open Images Dataset V4. IJCV, 128, 1956–1981.
5. Wu, H. et al. (2018). Look at Boundary: A Boundary-Aware Face Alignment Algorithm. CVPR 2018 (WFLW).
6. Sagonas, C. et al. (2016). 300 Faces in-the-Wild Challenge: Database and Results. Image and Vision Computing, 47, 3–18.
7. Zhu, X., Liu, X., Lei, Z. & Li, S. Z. (2016). Face Alignment in Full Pose Range: A 3D Total Solution. TPAMI (300W-LP).
8. Zhang, X. et al. (2015). It's Written All Over Your Face: Full-Face Appearance-Based Gaze Estimation. CVPR-W 2015 (MPIIGaze).
9. Weng, C.-H. et al. (2016). Driver Drowsiness Detection in a Simulation Environment by Using EEG-based Analysis. ACCV 2016 Workshops (NTHU-DDD).
10. Abtahi, S., O'Brien, M. & Sejkane, F. (2019). DROZY: A Dataset for Detection of Physiological Drowsiness. IEEE Access.
11. Ntavani, E. et al. (2018). Alertness and Drowsiness Detection: A Review (UTA-RLDD / vigilance stages).
12. Liu, Z., Luo, P., Wang, X. & Tang, X. (2015). Deep Learning Face Attributes in the Wild. ICCV 2015 (CelebA).
13. Khowaja, S. A. et al. (2020). MRL Eye Dataset (open/closed eye classification).
14. Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.
15. Redmon, J. & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.
16. Bettini, M. et al. (2019). Investigating Neck Pose Reliability for Gaze Estimation. ICMI 2019.
17. FastAPI documentation — https://fastapi.tiangolo.com/
18. Google Gemini API documentation — https://ai.google.dev/
19. Tailwind CSS / MDN Web Docs — getUserMedia, HTMLCanvasElement.toDataURL, WebSocket browser APIs.